

Using MikroDuino ADC SDK

MikroDuino ADC SDK Library



MikroDuino

Mikrotronics Pakistan

Analog-to-Digital Conversion using the MikroDuino SDK

Why Use the ADC Library?

In the previous chapter, we learned how the AVR Analog-to-Digital Converter works internally. We explored the ADC hardware, understood the conversion process, examined the hardware registers, and even wrote code that manipulated those registers directly.

Although direct register programming gives complete control over the hardware, it also has several disadvantages:

- The programmer must remember numerous register names and bit positions.
- Every project requires the same initialization code.
- Programs become difficult to read and maintain.
- Switching to another AVR device often requires register modifications.

The MikroDuino SDK solves these problems by providing a lightweight ADC driver that performs the repetitive register manipulation automatically while still giving the programmer full control over every important ADC feature.

Unlike many high-level frameworks, the MikroDuino ADC library does **not** hide the hardware. Every function corresponds directly to a hardware feature discussed in the previous chapter. Once you understand the AVR ADC, using this library becomes almost self-explanatory.

The philosophy is simple:

Simplify programming without sacrificing control.

Including the ADC Library

Using the ADC library requires only a single header file.

```
#include <mikroduino/adc.hpp>
using namespace MikroDuino;
```

The library automatically provides a global ADC driver object:

```
ADC_Driver
```

Since every AVR contains only one ADC peripheral, there is no need to create ADC objects manually. The SDK already provides a ready-to-use driver.

Initializing the ADC

The first step is enabling and configuring the ADC.

```
ADC_Driver.begin();
```

This single line replaces several register operations.

Internally it:

- enables the ADC
- selects AVCC as the voltage reference
- chooses a prescaler of 128
- configures right-adjusted 10-bit results

These defaults are suitable for nearly every beginner project running at 16 MHz.

Reading an Analog Channel

Reading an analog input becomes extremely straightforward.

```
uint16_t value = ADC_Driver.read(0);
```

This performs the complete conversion sequence:

1. selects ADC channel 0
2. starts the conversion
3. waits until conversion completes
4. reads ADCL and ADCH
5. returns the 10-bit result

Compare this with the equivalent register-based implementation from the previous chapter. What previously required several lines of register manipulation is now a single, self-explanatory statement.

Reading Multiple Channels

Changing channels is equally simple.

```
uint16_t sensor1 = ADC_Driver.read(0);  
uint16_t sensor2 = ADC_Driver.read(3);
```

```
uint16_t sensor3 = ADC_Driver.read(5);
```

The SDK automatically updates the channel multiplexer before starting each conversion. No manual register masking is required.

Choosing the Voltage Reference

Instead of manipulating the REFS bits inside ADMUX, the SDK provides an enumeration.

```
ADC_Driver.begin(ADCRef::AVCC);
```

Other available options are

```
ADCRef::AREF
```

```
ADCRef::Internal
```

These correspond directly to the hardware reference selections discussed earlier.

Changing the reference later is equally simple.

```
ADC_Driver.setReference(ADCRef::Internal);
```

Selecting the ADC Clock

The SDK replaces the ADPS bits with an easy-to-understand enumeration.

```
ADC_Driver.begin(  
    ADCRef::AVCC,  
    ADCPrescaler::DIV128  
);
```

Available prescalers are

```
DIV2  
DIV4  
DIV8  
DIV16  
DIV32  
DIV64  
DIV128
```

Instead of remembering binary values stored inside ADCSRA, the programmer simply chooses the desired clock divider.

Reading 8-bit Values

Many applications do not require the full 10-bit resolution.

Examples include

- LED brightness
- audio level meters
- joystick position
- menu navigation
- simple sensors

The SDK supports this mode directly.

```
ADC_Driver.begin(  
    ADCRef::AVCC,  
    ADCPrescaler::DIV64,  
    true  
);  
  
uint8_t value = ADC_Driver.read8(0);
```

Internally this enables left-adjust mode and reads only ADCH.

No register manipulation is required.

Continuous Sampling

Some applications need continuous measurements rather than manually starting each conversion.

Examples include

- oscilloscope projects
- audio sampling
- data logging
- motor control

The SDK provides Free Running mode.

```
ADC_Driver.begin();  
  
ADC_Driver.beginFreeRunning(0);
```

The ADC now performs conversions continuously in hardware.

The latest value can be obtained at any time.

```
uint16_t sample = ADC_Driver.resultFreeRunning();
```

When finished,

```
ADC_Driver.stopFreeRunning();
```

returns the ADC to normal operation.

Interrupt-Based ADC

Instead of waiting for each conversion to finish, the processor can continue executing other code. The ADC notifies the CPU whenever a conversion completes.

Enabling interrupts is simple.

```
ADC_Driver.enableInterrupt();
```

Likewise,

```
ADC_Driver.disableInterrupt();
```

disables them again.

The actual interrupt service routine is written exactly as in normal AVR programming.

```
ISR(ADC_vect)
{
    ...
}
```

The SDK simply handles the register configuration automatically.

Checking Conversion Completion

Polling applications can determine whether a conversion has completed.

```
if (ADC_Driver.conversionComplete())
{
    ...
}
```

After processing the result,

```
ADC_Driver.clearFlag();
```

prepares the ADC for detecting the next completed conversion.

Shutting Down the ADC

When analog conversions are no longer required, the ADC can be disabled.

```
ADC_Driver.end();
```

This reduces power consumption and is especially useful in battery-powered applications.

Summary

The MikroDuino ADC library provides an excellent example of the SDK's design philosophy.

Instead of hiding the hardware, it exposes the AVR ADC through a clean, consistent, and easy-to-understand interface. Every function maps directly to a hardware capability discussed in the previous chapter, allowing programmers to move effortlessly between high-level SDK programming and low-level register programming whenever necessary.

For beginners, the library eliminates repetitive register manipulation and reduces the likelihood of programming mistakes. For experienced developers, it preserves complete access to the underlying hardware while making application code significantly shorter, clearer, and easier to maintain.

Register Programming vs. MikroDuino SDK

Register Programming	MikroDuino SDK
Configure ADMUX	<code>ADC_Driver.begin()</code>
Configure ADCSRA	<code>ADC_Driver.begin()</code>
Select channel	<code>ADC_Driver.read(channel)</code>
Start conversion	<code>ADC_Driver.read()</code>
Wait for completion	handled automatically
Read ADCL/ADCH	return value of <code>read()</code>
Change reference	<code>ADC_Driver.setReference()</code>
Enable interrupt	<code>ADC_Driver.enableInterrupt()</code>
Free-running mode	<code>ADC_Driver.beginFreeRunning()</code>
Disable ADC	<code>ADC_Driver.end()</code>

This comparison would nicely conclude the transition from low-level AVR programming to productive application development using the MikroDuino SDK.

Practical ADC Applications

The best way to understand any peripheral is to use it in real applications. In this chapter, we will build several practical projects using the MikroDuino ADC library.

Each project demonstrates a different method of acquiring analog data.

- **Polling Mode** – the processor waits until the conversion finishes before continuing.
- **Interrupt Mode** – the processor performs other tasks while the ADC works in the background.
- **Free Running Mode** – the ADC continuously samples the analog signal automatically, producing a steady stream of measurements.

Although all three methods ultimately produce the same ADC readings, each has advantages depending on the application's requirements. By the end of this chapter, you will know when to use each approach.

Example 1 — Reading a Potentiometer (Polling Method)

Project Overview

In this project, a 10 k Ω potentiometer is connected to ADC channel 0. The program continuously measures the position of the potentiometer and prints the ADC value on the serial terminal.

This is the simplest and most common use of the ADC. Since the processor has nothing else to do while waiting for the conversion to finish, the polling method is perfectly suitable.

Hardware Required

- Arduino Uno / Nano
- 10 k Ω potentiometer
- USB cable

Connections

Potentiometer Pin	Connection
One end	+5V
Other end	GND
Wiper	ADC0 (A0)

Program Logic

The program performs the following steps repeatedly:

1. Initialize UART.
2. Initialize the ADC.
3. Read channel A0.
4. Print the ADC value.
5. Wait 200 ms.
6. Repeat forever.

Program

```
#include <mikroduino/adc.hpp>
#include <mikroduino/uart.hpp>
#include <util/delay.h>

using namespace MikroDuino;

int main()
{
    UART.begin(9600);

    ADC_Driver.begin();

    while (true)
    {
        uint16_t value = ADC_Driver.read(0);

        UART.print("ADC = ");
        UART.println(value);

        _delay_ms(200);
    }
}
```

Discussion

This is known as **blocking ADC conversion**.

The call

```
ADC_Driver.read(0);
```

does not return until the conversion has finished. During this time the CPU simply waits.

For slowly changing signals such as:

- potentiometers
- temperature sensors
- light sensors

this is perfectly acceptable and results in very simple code.

Example 2 — Temperature Monitoring (Interrupt Method)

Project Overview

Suppose we are building a temperature monitoring system using an LM35 temperature sensor.

Instead of waiting for every ADC conversion, the processor continues doing other work such as updating a display or communicating over UART. Whenever the ADC finishes converting, an interrupt automatically stores the latest reading.

This technique is commonly used in embedded systems where multiple tasks must run simultaneously.

Hardware

- Arduino Uno
- LM35
- LM35 output → A0

Program Logic

The program works as follows.

1. Initialize UART.
2. Configure ADC.
3. Enable ADC interrupt.
4. Enable global interrupts.
5. Start continuous conversions.
6. Continue performing other tasks.

7. Whenever a conversion completes, the interrupt stores the latest ADC value.

Flowchart

```
Initialize UART
    |
Initialize ADC
    |
Enable ADC Interrupt
    |
Enable Global Interrupts
    |
Start Free Running
    |
Forever
    |
Do Other Tasks
    |
ADC Interrupt Occurs
    |
Store ADC Result
```

Program

```
#include <avr/interrupt.h>
#include <mikroduino/registers.hpp>
#include <mikroduino/adc.hpp>
#include <mikroduino/usart.hpp>

using namespace MikroDuino;

volatile uint16_t adcValue;
volatile bool newSample = false;

ISR(ADC_vect)
{
    adcValue = ADC_Driver.resultFreeRunning();    // reading ADC
    clears ADIF and re-arms
    newSample = true;
}

int main()
```

```

{
    USART0.begin(9600);

    ADC_Driver.begin();           // AVCC, DIV128, 10-bit right-
aligned
    ADC_Driver.enableInterrupt(); // ADIE - fires ADC_vect on
every conversion
    ADC_Driver.beginFreeRunning(0); // ADC0, free-running (sets
ADTS=000 internally, starts first conversion)

    sei();

    while (true)
    {
        if (newSample)
        {
            uint16_t sample;
            ATOMIC_BLOCK_START;
            sample = adcValue; // 16-bit read must be atomic
w.r.t. the ISR
            newSample = false;
            ATOMIC_BLOCK_END;

            USART0.write("Temperature ADC = ");
            USART0.writeInt(sample);
            USART0.write("\r\n");
        }

        // Other processing can be done here.
    }
}

```

Discussion

Notice that the main program never calls

```
ADC_Driver.read()
```

Instead, every completed conversion automatically invokes

```
ISR(ADC_vect)
```

This allows the CPU to perform useful work while the ADC operates independently.

This technique is widely used in

- industrial controllers
- robotic systems
- communication equipment
- multitasking embedded applications

Example 3 — Audio Level Meter (Free Running Polling)

Project Overview

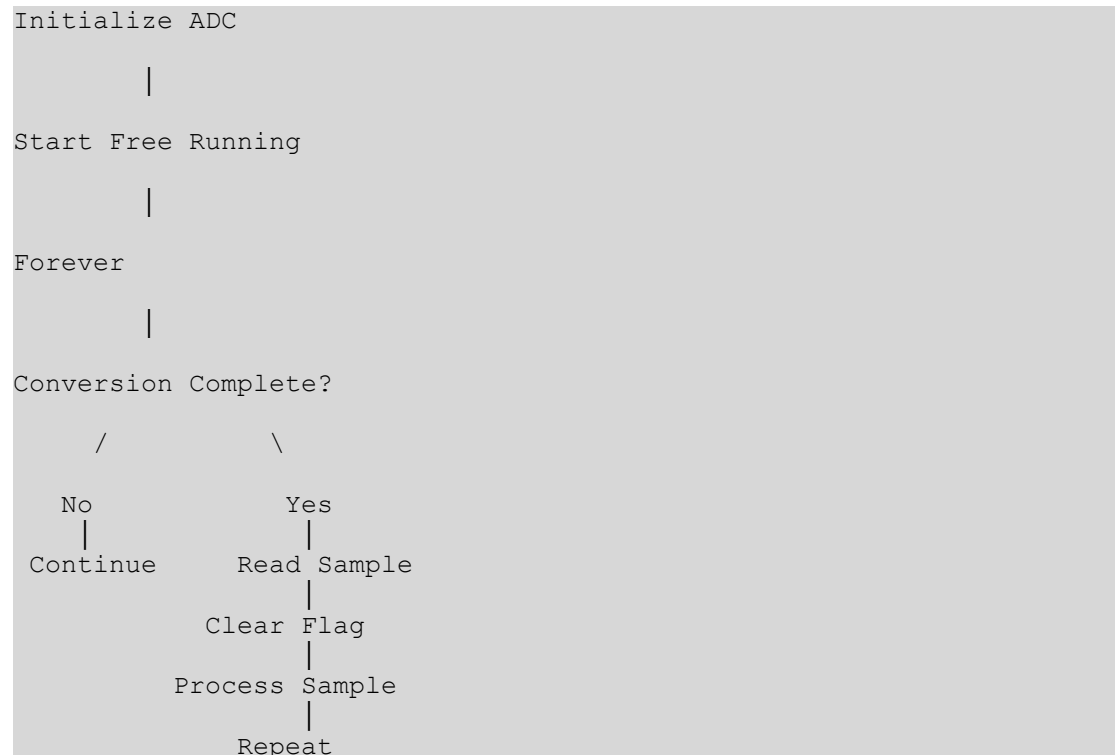
Suppose a microphone amplifier is connected to A0.

The ADC must continuously sample the signal at the highest possible rate. Waiting for each conversion manually would waste processing time. Instead, Free Running mode continuously performs conversions in hardware. The program simply checks whether a new sample has arrived.

Program Logic

1. Configure ADC.
2. Start Free Running mode.
3. Check whether a new conversion has completed.
4. Read the latest sample.
5. Process the sample.
6. Repeat forever.

Flowchart



Program

```
#include <mikroduino/adc.hpp>

using namespace MikroDuino;

int main()
{
    ADC_Driver.begin();

    ADC_Driver.beginFreeRunning(0);

    while (true)
    {
        if (ADC_Driver.conversionComplete())
        {
            uint16_t sample = ADC_Driver.resultFreeRunning();

            ADC_Driver.clearFlag();

            // Process the sample here.
        }
    }
}
```

Discussion

Unlike the first example, the program never starts conversions manually. The ADC hardware continuously performs conversions in the background. The CPU simply checks whether a fresh sample is available. This approach is ideal for

- digital oscilloscopes
- audio processing
- waveform analysis
- vibration monitoring
- data acquisition systems

where hundreds or thousands of samples must be collected every second.

Comparing the Three Techniques

Feature	Polling	Interrupt	Free Running
Easy to understand	★★★★★	★★★☆☆	★★★★☆
CPU waits during conversion	Yes	No	No
Suitable for slow sensors	Excellent	Excellent	Good

Feature	Polling	Interrupt	Free Running
Suitable for fast sampling	Poor	Very Good	Excellent
Multiple background tasks	Poor	Excellent	Excellent
Typical applications	Potentiometers, LM35, LDR	Robotics, control systems	Audio, oscilloscopes, data loggers

These three examples progressively introduce increasingly sophisticated ADC techniques while reinforcing that the **same ADC_Driver API** can be used in different ways depending on the application's performance requirements.

ADCSRB Register

```
ADCSRB &= ~0x07;
```

This line is clearing the **Auto Trigger Source (ADTS)** bits in the **ADCSRB** register so that the ADC uses **Free Running mode** as its trigger source.

Step 1: What is ADCSRB?

The **ADC Control and Status Register B (ADCSRB)** contains several control bits. The three least significant bits are called **ADTS2:ADTS0**.

ADCSRB Register

Bit:	7	6	5	4	3	2	1	0
	-	ACME	-	-	-	ADTS2	ADTS1	ADTS0

These three bits select **what event starts a new ADC conversion** when Auto Trigger mode (`ADATE = 1`) is enabled.

```
ADCSRB &= ~0x07;
```

Clears lower three bits of ADCSRB Register.

the register becomes

ADTS2 ADTS1 ADTS0

000

Everything else in ADCSRB remains unchanged.

Why is this necessary?

According to the AVR datasheet,

ADTS2:0	Trigger Source
000	Free Running

ADTS2:0	Trigger Source
001	Analog Comparator
010	External Interrupt INTO
011	Timer0 Compare Match
100	Timer0 Overflow
101	Timer1 Compare Match B
110	Timer1 Overflow
111	Timer1 Capture Event

Therefore,

```
ADCSRB &= ~0x07;
```

selects

```
ADTS = 000
```

which means

After each conversion finishes, the ADC hardware immediately starts the next conversion automatically.

No software intervention is required.

Why not simply write

```
ADCSRB = 0;
```

Because ADCSRB contains **other control bits** besides ADTS2:0, such as ACME.

If we wrote

```
ADCSRB = 0;
```

we would accidentally clear **every bit** in the register.

Using

```
ADCSRB &= ~0x07;
```

modifies **only** the trigger selection bits while preserving every other setting.

Note: why the manual ADCSRB &= ~0x07 line is gone

In the old snippet, the trigger source was being wired up by hand:

```
ADCSRB &= ~0x07;      // Free Running Trigger
```

```
ADC_Driver.enableInterrupt();
```



```
ADC_Driver.beginFreeRunning(0);
```

That direct register write cleared ADTS2:ADTS0 in ADCSRB to 000, which is the auto-trigger-source encoding for Free Running mode (see ADC.md §3, "ADTS source table"). It had to be done manually because there was no driver method that set up the trigger source — the caller was responsible for touching the raw register before calling beginFreeRunning().

The updated ADCDriver::beginFreeRunning() now does this itself, as documented in ADC.md §6 ("beginFreeRunning()"):

```
ADMUX = (ADMUX & 0xF0) | (channel & 0x0F); // select channel
```

```
ADCSRB &= ~(0x07 << ADTS0); // ADTS2:0 = 000 -> free running
```

```
ADCSRA |= (1 << ADATE); // enable auto-trigger
```

```
ADCSRA |= (1 << ADSC); // start first conversion
```

So the ADCSRB &= ~(0x07 << ADTS0) step is now baked into the function itself — beginFreeRunning() unconditionally configures the ADC for free-running mode (channel select, trigger source, auto-trigger enable, and kicking off the first conversion) as one atomic unit of behavior. There's no code path where you'd call beginFreeRunning() and not want ADTS cleared, so the redundant external write was folded into the driver rather than left as a caller obligation.

Leaving the old line in place wouldn't break anything (it's writing the same bits beginFreeRunning() already writes), but it's now dead redundancy — a leftover from when the driver only handled ADATE/ADSC and left trigger-source sele

Relation to the MikroDuino SDK

You may remember that the current implementation of `beginFreeRunning()` is:

```
void beginFreeRunning(uint8_t channel)
{
    selectChannel(channel);
    BITSET(ADCSRB, 0); // Sets ADTS0 = 1
    BITSET(ADCSRA, ADATE);
    BITSET(ADCSRA, ADSC);
}
```

Setting ADTS0 produces:

```
ADTS = 001
```

which actually selects the **Analog Comparator** as the trigger source, not Free Running mode. This appears to be a bug in the current implementation. The correct

implementation should ensure that all three ADTS bits are cleared before enabling auto-triggering:

```
FIELD_SET(ADCSRB, 0x07, 0, 0);    // ADTS = 000 (Free Running)
BITSET(ADCSRA, ADATE);
BITSET(ADCSRA, ADSC);
```

or simply:

```
ADCSRB &= ~0x07;    // Select Free Running trigger
```

This change would eliminate the need for users to manually clear the ADTS bits before calling `beginFreeRunning()`.